

```
@dataclass
```

```
class TCIDDecision:
```

```
    accepted: bool
```

```
    reasons: List[str]
```

```
    risk_score: float
```

```
def tcid_policy_engine(cred:
```

```
    CredentialRecord, policy: TrustPolicy,
```

```
    purpose: str) -> TCIDDecision:
```

```
    reasons = []
```

```
    risk_score = 0.0
```

```
    if cred.issuer not in policy.trusted_issuers:
```

```
        reasons.append("Untrusted issuer")
```

```
        risk_score += 0.5
```

```
    if not any(t in policy.required_types for t in cred.types):
```

```
        reasons.append("Credential type mismatch")
```

```
        risk_score += 0.3
```

```
if cred.id in policy.revoked_credentials:  
    reasons.append("Credential revoked")  
    risk_score += 0.8
```

```
if cred.assurance_level <  
policy.min_assurance_level:  
    reasons.append("Insufficient assurance  
level")  
    risk_score += 0.4
```

```
if purpose == "age_check" and "birthDate"  
not in cred.subject_claims:  
    reasons.append("Missing birthDate for  
age_check")  
    risk_score += 0.2
```

```
return TCIDDecision(  
    accepted=risk_score < 0.7 and  
len(reasons) < 2,  
    reasons=reasons,  
    risk_score=risk_score  
)
```